



Java Coffee — The Barista's Quest

OOP Project Proposal (Revised)

Nasywa Ramadhani
574106

Project Idea

Java Coffee — The Barista's Quest is a console-based turn-based RPG where a young barista embarks on a quest across three Indonesian coffee regions — Warung Jogja, Solo Heritage, and Dieng Highlands — battling rival brewers in coffee card battles, collecting rare beans, and working towards the title of Grand Master Barista. All interactions are direct object-to-object method calls: a Barista fights an Enemy, a CoffeeDeck holds CoffeeCards, an Inventory holds Items. Every game mechanic is a natural expression of an OOP concept.

Key Entities

 Characters & Enemies	 Coffee Cards	 Deck & Inventory	 Game World	 Items & Equipment	 Exceptions
Character (abstract) → Barista, SpecialistBarista Enemy (abstract) → RivalBarista, AngryCustomer, HealthInspector, CoffeeCritic, Brewmaster	CoffeeCard (abstract) → AttackCard, HealCard, DebuffCard, ComboCard	CoffeeDeck (composed of CoffeeCards), Inventory (composed of Items) — both owned by the Barista	Region (abstract) → WarungJogja, SoloHeritage, DiengHighlands Encounter → BattleEncounter, ShopEncounter, RandomEvent GameRule (interface) → player-authored rules at runtime	Item (abstract) → Ingredient (OatMilk, EspressoShot), Equipment (MokaPot), Collectible (RareBean)	EmptyDeckException InvalidBrewException OutOfStockException BattleOverException InvalidRuleException

OOP Concepts Checklist

	OOP Concept	How it appears in Java Coffee
✓	Abstraction	Abstract classes <code>Character</code> , <code>CoffeeCard</code> , <code>Region</code> define shared contracts. Interfaces <code>Brewable</code> and <code>Describable</code> declare methods without implementation.
✓	Encapsulation	All fields (health, stamina, deck) are private; accessed via public methods such as <code>takeDamage()</code> , <code>drawCard()</code> , <code>applyRule()</code> .
✓	Information Hiding	Enemy AI logic inside <code>act()</code> is private. The battle engine calls it without knowledge of the internal decision process. <code>BattleEngine</code> internals hidden from the World layer.
✓	Inheritance	<code>Barista</code> extends <code>Character</code> . <code>SpecialistBarista</code> extends <code>Barista</code> (3-level). <code>RivalBarista</code> , <code>AngryCustomer</code> , <code>HealthInspector</code> , <code>CoffeeCritic</code> , <code>Brewmaster</code> extend <code>Enemy</code> . <code>AttackCard</code> , <code>HealCard</code> , <code>DebuffCard</code> , <code>ComboCard</code> extend <code>CoffeeCard</code> .
✓	Subtyping	<code>Brewmaster</code> IS-A <code>Character</code> . The battle engine operates on <code>Character</code> references regardless of runtime type — Liskov principle applies.
✓	Multi-typing (Interface)	<code>ComboCard</code> implements both <code>Brewable</code> AND <code>Describable</code> . <code>Item</code> subtypes also implement <code>Brewable</code> , enabling reuse across unrelated hierarchies.
✓	Modularity	Five packages: <code>model</code> , <code>battle</code> , <code>world</code> , <code>items</code> , <code>exceptions</code> — each with a single, well-defined responsibility.
✓	Hierarchy	Multi-level: <code>Character</code> → <code>Enemy</code> → <code>RivalBarista</code> / <code>Brewmaster</code> ; <code>CoffeeCard</code> → <code>AttackCard</code> / <code>HealCard</code> / <code>DebuffCard</code> / <code>ComboCard</code> ; <code>Region</code> → <code>WarungJogja</code> / <code>SoloHeritage</code> / <code>DiengHighlands</code> .
✓	Composition	<code>Barista</code> HAS-A <code>CoffeeDeck</code> , HAS-A <code>Inventory</code> . <code>CoffeeDeck</code> HAS-A list of <code>CoffeeCards</code> . <code>Region</code> HAS-A list of <code>Encounters</code> . No inheritance coupling.
✓	Reuse	<code>Brewable</code> interface reused across <code>CoffeeCard</code> and <code>Item</code> hierarchies. Battle engine logic reused for all <code>Character</code> subclasses. <code>Inventory</code> reused for any item type without duplication.

✓	Polymorphism (Inclusion)	<code>enemy.act()</code> and <code>card.use(target)</code> produce different behaviour at runtime depending on the concrete type of each object.
✓	Polymorphism (Overloading)	<code>CoffeeDeck</code> defines three versions of <code>draw()</code> : no arguments (draw one card), <code>draw(int n)</code> (draw n cards), <code>draw(CardType filter)</code> (draw only cards of a given type). Java picks the right version based on what is passed in.
✓	Polymorphism (Parametric)	<code>Inventory<T extends Item></code> and <code>Deck<T extends CoffeeCard></code> — generic containers reused with type safety.
✓	Polymorphism (Coercion)	<code>Ingredient</code> cast to <code>Brewable</code> when passed to the brew engine — implicit widening to interface type.
✓	Extensibility	New enemies, cards, regions, encounters, and — critically — new <code>GameRule</code> types are added without modifying existing classes. The player can author entirely new rule behaviour at runtime via the console rule syntax — open/closed principle applied to game logic itself, not just code structure.
✓	Exception Handling	Custom exceptions thrown and caught during gameplay: <code>EmptyDeckException</code> , <code>InvalidBrewException</code> , <code>OutOfStockException</code> , <code>BattleOverException</code> , <code>InvalidRuleException</code> .

Extensions & Customizability

Feature	Description
Player-Authored Rules (GameRule)	Rather than selecting among predefined difficulty options, the player authors brand-new rules at game start through a console mini-syntax: <code>IF <condition> THEN <effect></code> (e.g. <code>IF player.health < 20 THEN player.gain("EspressoShot")</code>). Each parsed line becomes a <code>GameRule</code> object — <code>ConditionalBonusRule</code> , <code>PeriodicDrawRule</code> , <code>BannedCardRule</code> , etc. — implementing <code>boolean appliesTo(GameEvent)</code> and <code>void execute(GameEvent)</code> . The player's custom rules are stored in <code>List<GameRule></code> inside <code>RuleSet</code> and checked every turn by <code>BattleEngine</code> — new rule behaviour is genuinely created by the player, not chosen from existing options.
Dynamic Rule Mutation	The <code>CoffeeCritic</code> enemy can inject its own temporary <code>GameRule</code> mid-battle (e.g. a rule that bans <code>HealCards</code> for 2 turns), demonstrating that rules are first-class objects that can be added at runtime by any entity, not just the player at setup.
Status Effects	<code>Burned</code> , <code>Steamed</code> , <code>Jittery</code> — managed as a <code>List</code> on <code>Character</code> , applied and ticked down each turn via polymorphic <code>tick()</code> calls.
Procedural Encounters	Each <code>Region</code> shuffles its encounter pool per run. <code>Encounter</code> subtypes — <code>BattleEncounter</code> , <code>ShopEncounter</code> , <code>RandomEvent</code> — are all polymorphic, resolved by the <code>WorldManager</code> without type-checking. Regions follow an Indonesian coffee journey: Warung Jogja → Solo Heritage → Dieng Highlands.
Combo System	Consecutive card plays of the same type trigger a combo bonus effect, tracked by <code>BattleEngine</code> state, adding strategic depth without a GUI.
RunSummary & Score Log	A <code>RunSummary</code> object records every action taken during the run and prints a structured report at game end — score, turns survived, cards played, enemies defeated.

Frameworks & Libraries

Pure Java (JDK 17+), no external libraries. IDE: NetBeans. Build: Ant (default NetBeans build). The game runs entirely in the console. All inter-entity interactions are direct in-memory method calls — no DB, no web interface, no third-party API.

I look forward to your acknowledgment. Thank you!